

VR-TAK Plugin Developer Guide

5.4.0

2025-03-24
Setup and overview



2025-03-24

Table of Contents

VR-TAK Plugin Developer Guide 5.0.0	1
Setup and overview	1
Overview.....	3
Unreal Editor Basics	3
Unreal Plugin Overview	3
Native / C++	4
Blueprints.....	5
Branch, sequence, cast.	6
Functions vs. Events	7
Some helpful hotkeys	8
Environment Setup.....	8
Creating a New Plugin	9
VR-TAK Example Plugin Setup	11
PluginName/BP/Core/BP_PluginName_PluginActor	11
PluginName/BP/Core/BP_PluginName_PluginActor::EventBeginPlay	11
PluginName/BP/Core/BP_PluginName_PluginActor::EventHandleCOTMessageReceived	11
PluginName/BP/Data/EntityHandlers/BP_PluginName_EntityHandler::PageInEntity.....	11
PluginName/UI/ContextMenu/CO_PluginName_Delete::GetEntityInfo	13
PluginName/UI/ContextMenu/CO_PluginName_DetailMenu::Execute	14
PluginName/UI/ContextMenu /CO_PluginName_PluginMenu::Execute.....	14
PluginName/UI/PluginMenu/WBP_PluginName_PluginMenu::HandleCOTEvent.....	15
Giving your plugin a custom tab on the ribbon	16
Blank VR-TAK Plugin Setup	16
Compiling a plugin	16

Overview

This document gives an Unreal Engine primer, then describes how to create a new VR-TAK plugin.

Unreal Editor Basics

The following is a brief explanation of some core features in Unreal Engine.

When you open the editor, you are presented with a large viewport into the virtual world. Above this viewport is a green “Play” button which allows you to run a development version of VR-TAK. Below the viewport is the Content Drawer, which you can summon using Space + Ctrl to see project assets.

VR-TAK is made in the Unreal Engine. This manual will give you a brief overview of how to use the Unreal Editor to set up a VR-TAK Plugin. Full Unreal documentation is online at <https://docs.unrealengine.com/>

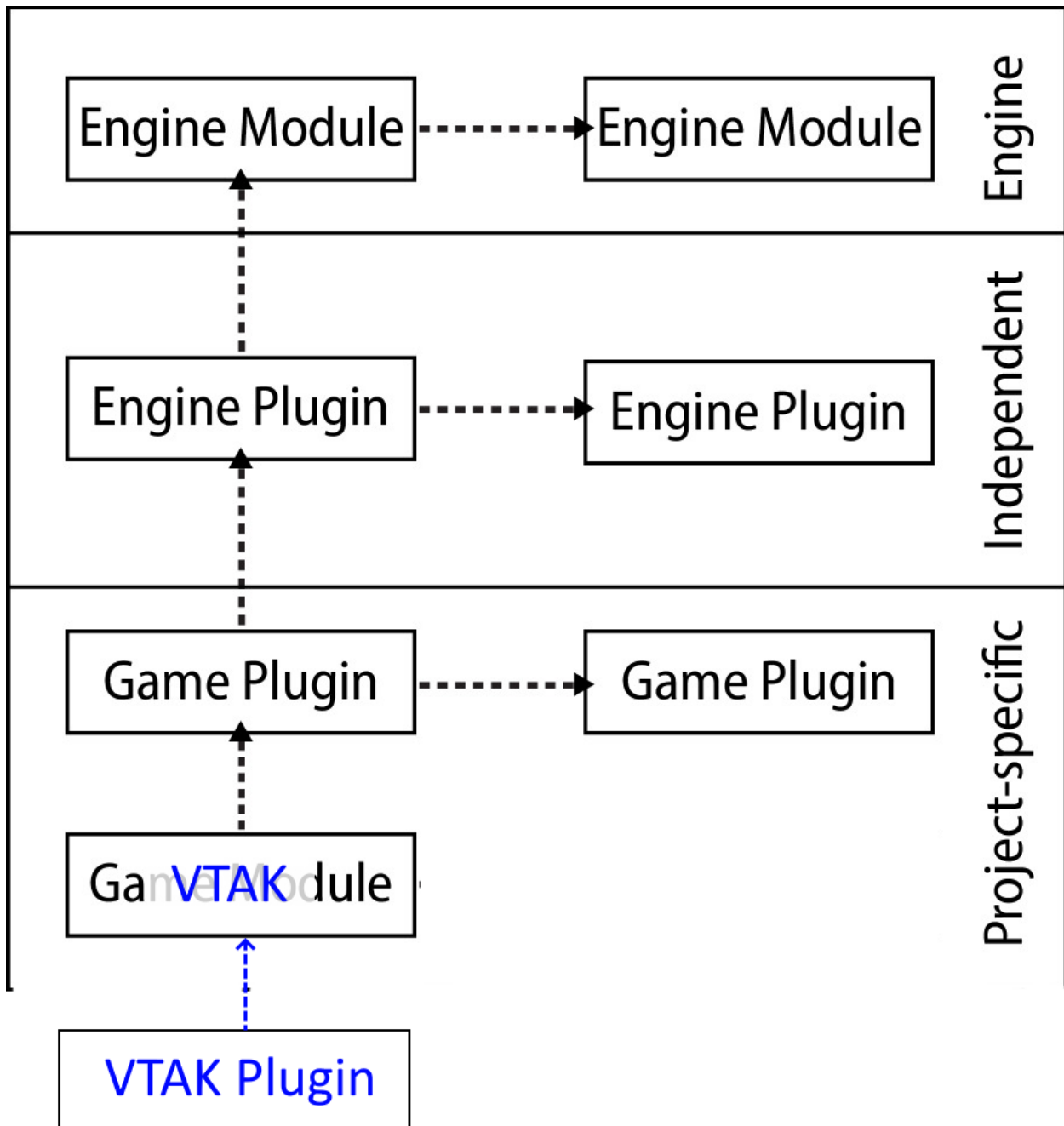
VR-TAK is made using Unreal Engine 5. Most of the existing documentation for Unreal Engine 4 is still accurate and up-to-date for Unreal Engine 5.

Unreal Engine an open-source commercial game engine built in C++. The engine's source code is bundled with the VR-TAK Plugin SDK. VR-TAK core code only includes headers and build artifacts.

Unreal Plugin Overview

Documentation can be found here which gives a basic overview of Plugins in Unreal Engine: <https://docs.unrealengine.com/5.0/en-US/plugins-in-unreal-engine/>

VR-TAK Plugins are dependent on the “Game” Layer, which is VR-TAK itself. A modified diagram from the above documentation shows where VR-TAK Plugins fit into the hierarchy:



Native / C++

As a plugin developer, you'll use the Unreal Editor and optionally C++ (also known as native). As you do so, you'll use Unreal's reflection system to expose your objects and properties in blueprints and in the application. Visual Studio is the recommended IDE for this type of development.

<https://docs.unrealengine.com/4.26/en-US/ProgrammingAndScripting/ProgrammingWithCPP/>

<https://docs.unrealengine.com/4.26/en-US/ProgrammingAndScripting/ProgrammingWithCPP/IntroductionToCPP/>

VR-TAK's header files are stored in Game/Source/Game/...

They rely on Unreal Engine code, in Engine/Source/...

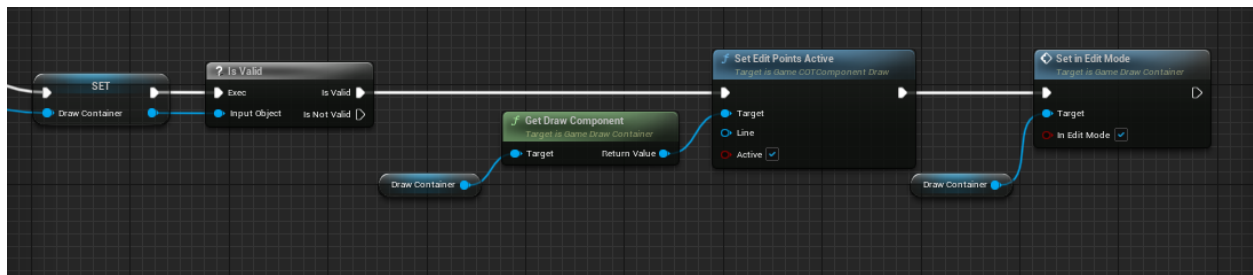
Plugins are primarily developed in Game/Plugins/<PluginNameFolder>/

Blueprints

A Blueprint (BP) is an object made and editable in the Unreal Editor. Blueprints use a visual scripting language in which you can place nodes representing functions and variables on a node graph to control the behavior of an object.

<https://docs.unrealengine.com/4.26/en-US/ProgrammingAndScripting/Blueprints/>

Open a blueprint using the content browser to view variables and functions. Variables and functions are listed on the left side of a blueprint, and under "Class Defaults"

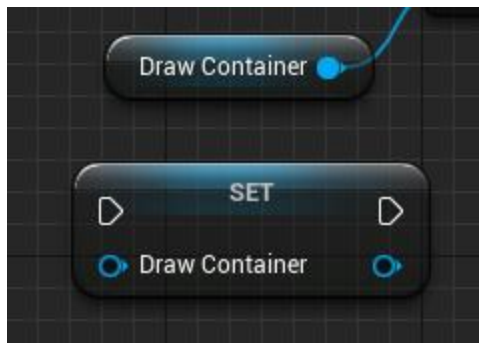


Nodes are connected using wires by their pins. They should be placed such that logic goes left to right, and sometimes top to bottom.

The white wire is the Execution Wire, it determines the flow of logic. It starts at some origin and continues calling all functions in order until it either ends or hits a Return node.

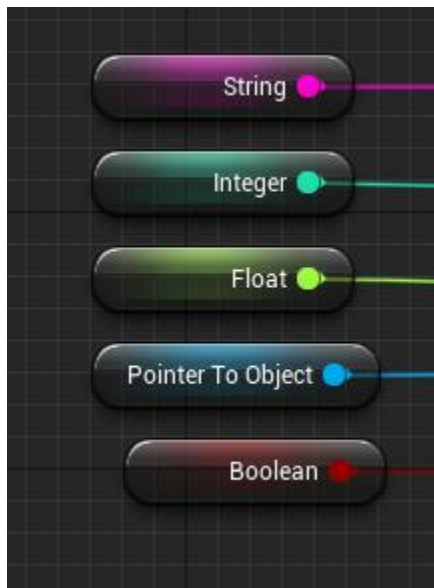
Events are similar to functions and are displayed on the Event Graph. Functions are each displayed in their own graph and will open when double-clicked.

Variables can be accessed using Get and Set nodes. Get nodes do not have Execution pins and should be placed close to where they are used.



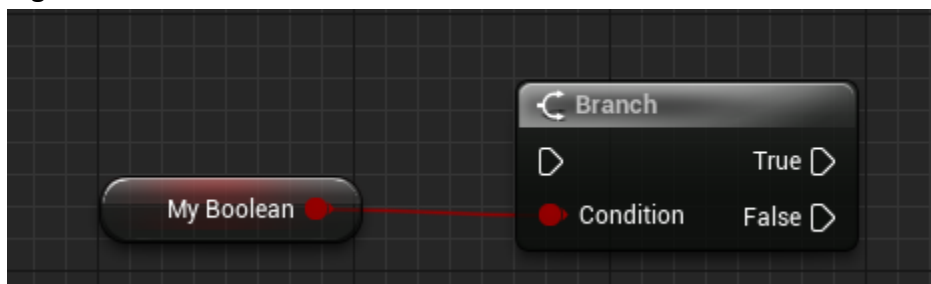
Like in C++, variables can be defined on the class or locally within a function. Variables local to a function cannot be used outside of that function.

Colors on pins denote data types. Some common data types:

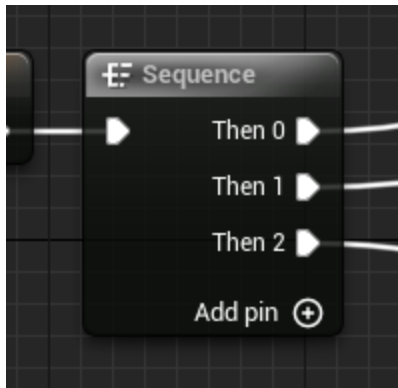


Branch, sequence, cast.

A Branch is like an if statement; it has two outgoing execution nodes and will send the flow of logic towards the one that matches the connected boolean's value.

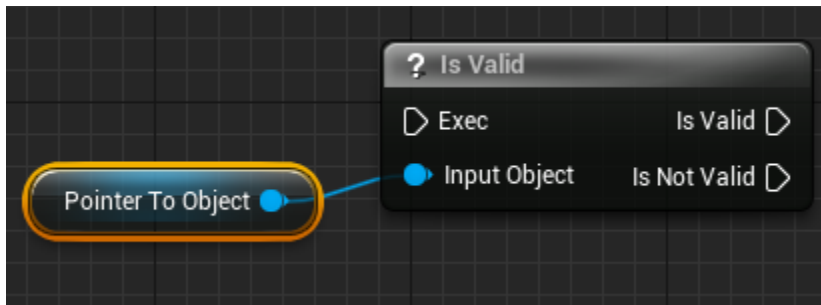


A sequence will run all outgoing execution wires, one after another.



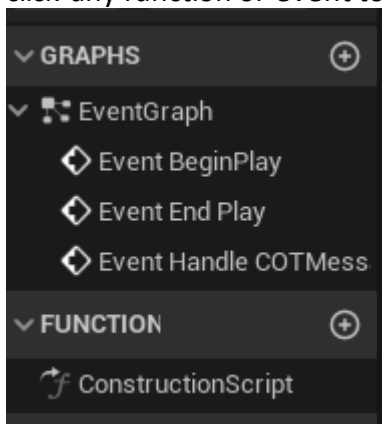
Cast nodes behave about the same as Casting in C++. In blueprint, you must cast your references to the appropriate class before you can call class-specific functions on them.

Additionally, be aware that Object pointers can be null. If there is ever a chance an object pointer is invalid, run the “Is Valid” function on it before using it.



Functions vs. Events

After opening a blueprint, a list of graph Events and Functions is shown on the left side. Double click any function or event to navigate to it



Functions and events are very similar in that both handle flows of logic, similar to C++ functions.

Only Functions can have return values and local variables.

Only Events can have Timelines, Delay nodes (which can make things happen after a certain amount of time has passed), and support for online replication.

Some helpful hotkeys

Ctrl + S = Save the current level or blueprint (just the one on screen right now)

Ctrl + Shift + S = Save all open blueprints

Ctrl + F = Search the current blueprint

Ctrl + Shift + F = Search all blueprints

Ctrl + P = "Play" / run VR-TAK in the Unreal Editor Play-In-Editor mode.

Ctrl + Tab = Quickly Navigate between open Blueprints

After using the left mouse button to select a group of nodes in the blueprint graph editor:

C = Create a comment around the selected nodes

W = Straighten the connections between selected nodes

Double Click on an execution wire = Add a reroute node at the mouse position

To remove a wire from a node:

Hold Alt and click where the wire is connected to a node

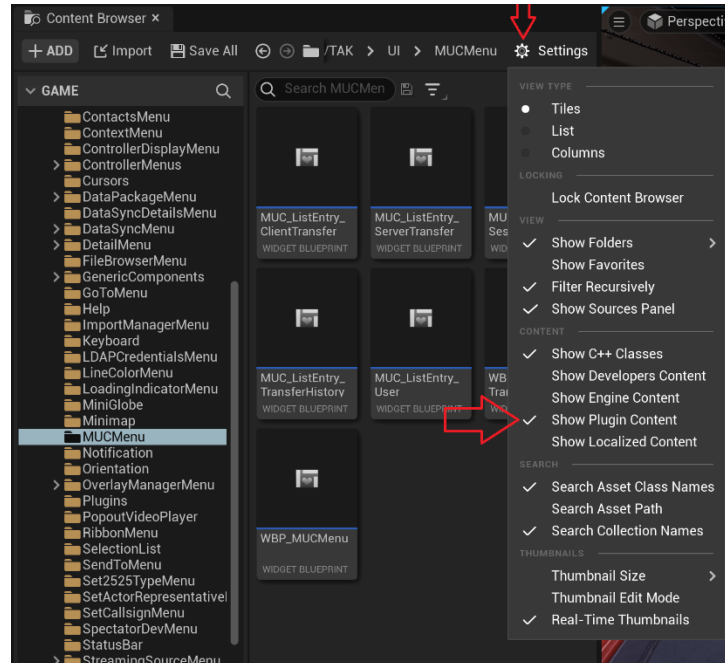
To move a wire from a node:

Hold Ctrl and click where the wire is connected to a node

Environment Setup

1. Install Visual Studio 2022
 - a. <https://docs.unrealengine.com/en-US/Programming/Development/VisualStudioSetup/index.html>
2. Unzip the VR-TAK Plugin SDK 7zip file
3. Run directx-Jun2010_redist and UE5PrereqSetup_x64 from the Prereqs folder
 - a. After running the Jun2010 redistributable, navigate to the temp folder and run DXSETUP.exe
4. Run the [root]/GenerateProjectFiles.bat file
5. Open [root]/UE5.sln in Visual Studio 2022
6. At the top of Visual Studio, make sure Solution Configuration is Development Editor and Solution Platforms is Win64.
7. In Visual Studio's Solution Explorer, navigate to the Programs folder, right-click on UnrealBuildTool and click Build. Do the same for UnrealHeaderTool. You should only need to build these tools once.
8. In Visual Studio's Solution Explorer, right-click on UE5 project and choose Build, this builds the Engine layer. You should not change Engine layer code and should only need to build the Engine layer once.
9. Run [root]/build_dev.bat to create build artifacts for VH_Example plugin and make sure the build pipeline is working.

10. Create a shortcut for [root]/Engine/Binaries/Win64/UE4Editor.exe. Then right-click and view "Properties". Make sure the "Shortcut" tab is selected and then inside the "Target" textbox add the following parameters: Game -log -skipcompatible -dev
11. Double-click on shortcut (it'll take a while to build shaders the first time)
12. To view plugin content, make sure View Options -> Show Plugin Content is checked.



Creating a New Plugin

At the very top of the Unreal Editor, choose File->Create Plugin. After doing so, a popup window will appear giving you two options to select from.

1. **VR-TAK_ExampleTemplate:** The easiest way to get started making a VR-TAK plugin with some basic plugin functionality. Adds one Tool and one Menu to the ribbon, which can be modified, copied, or inherited from to enable custom functionality. Due to the how the Unreal Plugin Template system works, you'll need to follow some extra steps (shown below) to enable full functionality when using this option.
2. **VR-TAK_BlankTemplate:** Recommended for developers experienced with Unreal and VR-TAK. Contains the bare minimum amount of code to make a VR-TAK Plugin.

After selecting the type of plugin to create input a plugin name, author and description; leave "Is Engine Plugin" and "Show on Startup" unchecked. Next press "Create Plugin".

In this document, PluginName represents the name of your plugin.

Both templates require some additional setup. Navigate to your VR-TAK project's directory, and open the following file in a text editor:

Game/Plugins/PluginName/PluginName.uplugin [specify file path for save]

You will see basic information about your created plugin. After the last attribute listed, add a comma and then start a new line. Paste the following on the new line, but still inside the outermost braces:

```
"Modules": [
    {
        "Name": "PluginName",
        "Type": "Runtime",
        "LoadingPhase": "Default"
    },
    "Plugins": [
        {
            "Name": "VH_COT",
            "Enabled": true
        },
        {
            "Name": "VH_Plugins",
            "Enabled": true
        }
    ]
}
```

Don't forget to replace all instances of "PluginName" with the name of your custom plugin throughout this document.

Close the Unreal Editor, and then run "GenerateProjectFiles.bat" found in your root project directory. Then run build_dev.bat [elaborate more?]

Open the Visual Studio project if it's not already open, build and run.

Once the Editor is open, use Ctrl+Space to open the Content Drawer on the bottom of the editor.

If you don't like the Content Drawer's auto-close behavior, open a permanently expanded Content Browser. At the top of the Unreal Editor, click Window->Content Browser->Content Browser 1.

On the left side, scroll down through the folder tree until you find a folder for your plugin.

If you cannot find your plugin, Click the Settings cog in the top right of your Content Drawer/ Browser and check the option for "Show Plugin Content"

Once you select the folder for your plugin on the left side, you should see two folders: BP and UI. Navigate into a folder by double clicking. Navigate back up by clicking on a higher level folder name shown on the top of the Content Browser.

VR-TAK Example Plugin Setup

In the section below, if a blueprint you need to select in a node does not show in that node's dropdown menu, navigate to the blueprint and open it. Afterwards, it should show in the dropdown menus. You should only need to do this once.

PluginName/BP/Core/BP_PluginName_PluginActor

- In the Variables section, set the default value of ExamplePluginMenuID and ExampleCubeController to a unique ID.

PluginName/BP/Core/BP_PluginName_PluginActor::EventBeginPlay

- Within the "Add menu Button..." Section, find the node "Create Image Button Menu Widget" and set the "Menu Class" to WBP_PluginName_PluginMenu*
- Within the "Add CubeDropper button to Ribbon" Section, find the node "Create WBP Image Button Controller Widget" and set the "Controller Class" to BP_PluginName_CubeController*
- Within the "Registry Entity Handler" section, connect the Construct Node to the sequence node to its left and set its Class using the dropdown to BP_PluginName_EntityHandler*
- Compile and Save

**Remember, replace PluginName with your plugin's name throughout these instructions*

PluginName/BP/Core/BP_PluginName_PluginActor::EventHandleCOTMessageReceived

- At the end of the event, connect the unlabeled (top) output execution pin of "Cast To Game_COTManager" to the "RegisterVehicleClass" node. Do not connect the Cast Failed node.
- On the "RegisterVehicleClass" node, open the Vehicle Blueprint pin dropdown menu and select "BP_PluginName_SimpleHeli"
- Compile and Save

PluginName/BP/Data/EntityHandlers/BP_PluginName_EntityHandler::PageInEntity

- Find the SpawnActor Node. Connect its output execution pin to the Cast To Game_WorldInteractableActor Node.

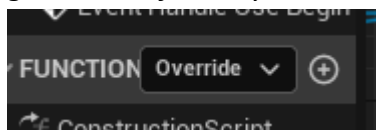
The PageInEntity function runs when an item is loaded from the database into the displayed virtual world/TOC.

- Find the "L Cube" getter node on the far right of the function. Click and hold on the blue pin on its right side, drag your mouse to the right and release. A menu will appear showing possible functions to run on the L Cube reference. Type "Cast to cube"
 - Click the entry with your plugin name, and a node will be created to cast the variable to the cube actor which comes in your newly created plugin
 - Connect the "AddObjectstoEntity" output execution pin node to your newly created "Cast to cube" node
- Drag out from the blue pin on the right side of the Cast node, and type "Set Entity UID" in the menu that appears. Click the option that shows up under the "Call Function" category.
- From the pink Entity UID variable node, drag a wire into the "Entity UID" input pin on Set Entity UID
- Compile and Save

Every COT entity has a unique UID. This function sets the UID on the actor which just spawned into the world to match the database's UID for that entity.

PluginName/BP/Vr/Controllers/BP_PluginName_CubeController::EventHandleUseBeginWorld

- In the purple Set Actor Class node, use the dropdown to set the Actor class to BP_PluginName_Cube. Connect the execution wire to it on its left side.
 - *The Event Handle Use Begin World event runs when the controller is "used" (trigger pulled in VR, mouse clicked in Desktop) on terrain or a model in the world.*
 - *To override other useful functions and events, hover your mouse over the word "Functions" and click the "Override" Dropdown. Looking at the list will give you a good idea of what you can do with a custom controller in VR-TAK.*



- Find the blue SpawnedActor variable node on the right side of the function. Drag out from it to create a Cast to BP_PluginName_Cube. From there, drag out from the blue pin and call the Set Entity UID function under the "Call Function" category.
 - Drag out again from the same blue pin and call the "Insert" function in the "Call Function" category. Make sure the "Insert" node is executed after the "SetEntityUID" node by connecting the "SetEntityUID" node's output execution pin to the "Insert" node's input execution pin

- Every COT must have a unique identifier. This is used for both database storage and sending COT to other TAK users.
- There is a node "New Guid" connected to a conversion node, which has a pink output pin. Connect that pink pin to the Entity UID Pin on Set Entity UID.
- Connect the execution wire through all the nodes you created
- Compile and Save

The Insert function adds the newly created cube to VR-TAK's internal SQLite database, allowing it to persist even when it's out of sight. When the user moves to a new part of the world, the database is queried for items within that geographic area. When the program is closed, the database is preserved.

In VR-TAK, when you "click" on an entity in the world, a context menu opens showing several options. These context option objects each specify the behavior of a context option. They are prefixed with CO

PluginName/UI/ContextMenu/CO_PluginName_Delete::GetEntityInfo

- Drag out from the "Context" pin on the leftmost node, and make a node for "Cast to BP PluginName Cube. Connect its input execution pin to the beginning of the function.
- From the cast node, connect the Cast Failed pin to the Return Node which is connected to "Parent: Get Entity Info"
- From the blue As BP PluginName Cube pin, drag out and call the cube's GetEntity UID Function.
- Repeat the last step, but this time call the Get Class UID function from the "Call Function" category.
- Right click, and make a Return Node by typing "Add Return Node" and choosing the only option.
- Compile and Save

Return Nodes are the blueprint equivalent of a return statement in a C++ function. The input pin(s) on the node represent the function's return type.

- Connect the execution wire from the Cast node, to the Get Entity UID node, from the Get Entity UID node to the Get Class UID node, then finally from the Get Class UID node into the Return Node.
- Finally, connect the pink string nodes from Get Entity UID and Get Class UID into the corresponding return values on the return node, and tick the box for "Success"

- Consider adding reroute nodes to make the code more readable (it's best to avoid overlapping wires and nodes when possible.)
- Compile and Save

By double clicking on any wire, you can create a "reroute node", which is an intermediary pin that holds the wire in place. It is helpful for keeping functions clean by rerouting wired around other nodes, rather than letting the execution wire run behind other nodes.

PluginName/UI/ContextMenu/CO_PluginName_DetailMenu::Execute

- Drag out from "Interaction Data" pin on the far left "Execute" node. From it, make a "Get Specific Context" node.

Interaction Data is a struct which has several attributes, including "Specific Context," which holds a reference the object interacted with. For example, if the VR-TAK user points at a cube and clicks on it, the Specific Context references that cube.

- Cast that node to BP_PluginName_Cube by dragging out from its right side and typing "Cast to PluginName Cube" If not done already, connect the 0 slot of the Sequence node to your "Cast to PluginName_Cube" node.
- Drag out from the As BP PluginName Cube pin and make a "Get Entity UID" node in the "Call Function" category.
- Connect that node to the Cast node's top execution pin and connect both its output pins to the Set L Entity UID node.
- Compile and Save

PluginName/UI/ContextMenu /CO_PluginName_PluginMenu::Execute

- From the Specific Context variable node on the far left, drag out and type "Cast To BP_PluginName_Cube" to cast the variable.
- Drag out from As BP_PluginName_Cube, and call its function "Activate Particle" by typing and then choosing it in the menu that appears.
- Set the Duration to 5 seconds
- Connect the execution wire from Execute to your Cast To BP_PluginName_Cube, input and from your Cast To BP_PluginName_Cube (the top one, not the cast failed output pin) to Activate Particle.
- Connect the Activate Particle Node to the Get Context Menu Node
- Compile and Save

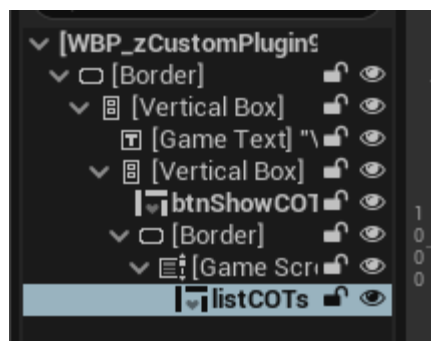
PluginName/UI/PluginMenu/WBP_PluginName_PluginMenu::HandleCOTEvent

- Open the “Graph” version of this BP by selecting “Graph” in the top right corner of the screen, then open the HandleCOTEvent function
- Connect the Branch node's "True" output to the "Create Entry by Class" node.
- On the Widget Instance input pin, use the dropdown to select WBP_PluginName_ListItem_COT

The Plugin Menu is a Widget Blueprint. It makes use of Unreal Engine's UMG system, which is used for making user interfaces. General information on UMG can be found in the official documentation <https://docs.unrealengine.com/4.26/en-US/InteractiveExperiences/UMG/UserGuide/>

To get from the UMG Designer to the Node Graph, click "Graph" on the top right after opening the menu widget blueprint

If you move back to the "Designer" view, you can see the listCOTs vertical list in the widget blueprint heirarchy. This is the UI element you are referencing in the Blueprint code with the "List COTs" getter node



It is not necessary to edit the hierarchy to complete this setup, go back to the Node Graph.

- Drag out from Return Value of the “Create Entry by Class” Node and call the function "Set COTEvent."
- Connect the blue wire running below the “Create Entry by Class” function to the Cot Event pin on Set COTEvent

That's it. If you go back to the main editor window by minimizing or closing the blueprint windows, and then press the Play button, you should be able to see your new menu and tool within VR-TAK, in a tab named "Plugins" in the VR-TAK Ribbon. The VR-TAK Ribbon is toggled on and off with the [m] key.

The Controller places interactive COT cubes in the virtual world.

The menu can display all incoming COT Messages.

Giving your plugin a custom tab on the ribbon

If a plugin adds a small number of ribbon buttons (<7), it is recommended to leave them nested under the "Plugins" tab on the ribbon.

If a plugin adds a significant number of ribbon buttons (7+), it should have its own tab.

The plugin actor (`PluginName/BP/Core/BP_PluginName_PluginActor`) is what adds your custom tool and menu to the ribbon. Open it and navigate to Event BeginPlay.

On the node Create Image Button Menu Widget, observe the value of the "Path" input pin. Replace "Plugins" with the name of your custom tab.

Repeat for the node Create WBP Image Button Controller Widget.

Find the EventHandleCOTMessageReceived event, and locate the "Make SOverlayCategory" node. Edit the ID and Display name to match your desired custom tab name, and optionally assign an icon.

Blank VR-TAK Plugin Setup

The blank VR-TAK Template has no functionality, but still contains the core components necessary for a VR-TAK Plugin.

BP/Core/BP_PluginName_PluginActor::Event BeginPlay

- Connect the BeginPlay node to the Construct node to its right. For the class on the Construct node, choose `BP_PluginName_EntityHandler`

Compiling a plugin

To compile a plugin, edit [root]/build_dev.bat and replace VR-TAK_Example with your plugin's name. Then run [root]/build_dev.bat